

## Using the STAR-System RMAP Target API for the PXI Interface Board

STAR-Dundee's PXI SpaceWire Interface with RMAP Target provides access to four RMAP targets and 1 Gbyte of DDR3 that can be divided between the targets using address offset registers, allowing each target to have a separate segment of memory. Incoming RMAP commands can be authorised either automatically, using a set of authorisation parameters, or manually by the user application. Additionally, the user application can be notified of RMAP command activity and authorisation requests through notification packets sent on the PXI bus.

The API provided with STAR-System provides functions to control the address offsets for each target, get the status of the RMAP targets, set the authorisation mode, set the authorisation parameters, enable and disable each RMAP target, perform manual authorisation, enable and disable notifications, register and unregister notification listener functions, and read and write to the RMAP target memory.

This application note explains how the RMAP Target API can be used to configure the address offsets for each RMAP target, set up automatic RMAP command authorisation, perform manual RMAP command authorisation, enable and receive notifications using a notification listener function, and read and write to memory. Further information on the API and the functions provided can be found in the STAR-System documentation.

### Configuring the Address Offsets

The RMAP Target API provides the following functions to control how the target memory is addressed by the four RMAP targets:

- [RMAP\\_TARGET\\_PXI\\_IF\\_setAddressOffset\(\)](#),  
[RMAP\\_TARGET\\_PXI\\_IF\\_getAddressOffset\(\)](#)

These functions can be used to set and get the address offset for each RMAP target. The set function takes a device identifier, a target index and an address offset, which is measured in 1 Mbyte blocks. The get function takes a device identifier, a target index and a pointer to a value which will be updated with the address offset.

The example below shows how to set the address offset for each target so that the memory regions for targets 0, 1, 2 and 3 start at 0x00000000 (0 Mbytes), 0x01000000 (256 Mbytes), 0x02000000 (512 Mbytes), and 0x03000000 (768 Mbytes), respectively.

Note that the example assumes the application already has the device ID of a PXI SpaceWire Interface with RMAP Target board which is stored in the "deviceId" variable.

```
U32 offset;
int setSuccess, getSuccess;
int i;

for(i = 0; i < 4; ++i)
{
    /* Set the address offset */
    setSuccess = RMAP_TARGET_PXI_IF_setAddressOffset(deviceId, i, i * 256);
    if(!setSuccess)
```

```

{
    printf("Setting the address offset for target %d failed.\n", i);
    continue;
}

/* Get the address offset */
getSuccess = RMAP_TARGET_PXI_IF_getAddressOffset(deviceId, i, &offset);
if(!getSuccess)
{
    printf("Getting the address offset for target %d failed.\n", i);
    continue;
}

/* Compare the address offset with the expected value */
printf("The address offset for target %d was %sset correctly.\n",
    i, offset == (i * 256) ? "" : "not ");
}

```

## Setting Up Automatic RMAP Command Authorisation

The RMAP Target API provides the following functions for setting the automatic RMAP authorisation parameters:

- [RMAP\\_TARGET\\_PXI\\_IF\\_setAuthLogicalAddressRange\(\)](#),  
[RMAP\\_TARGET\\_PXI\\_IF\\_getAuthLogicalAddressRange\(\)](#)

These functions can be used to set and get the authorised logical address range to authorise or reject an incoming RMAP command based on the target logical address value in the RMAP command header. The set function takes a device identifier, a target index, and the values of the lowest and highest logical addresses authorised. The get function takes a device identifier, a target index, and two pointers to values which will be updated with the lowest and highest logical addresses authorised.

- [RMAP\\_TARGET\\_PXI\\_IF\\_setAuthProtocolId\(\)](#),  
[RMAP\\_TARGET\\_PXI\\_IF\\_getAuthProtocolId\(\)](#)

These functions can be used to set and get the authorised protocol ID to authorise or reject an incoming RMAP command based on the protocol ID value in the RMAP command header. The set function takes a device identifier, a target index and the value of the protocol ID authorised. The get function takes a device identifier, a target index and a pointer to a value which will be updated with the protocol ID authorised.

- [RMAP\\_TARGET\\_PXI\\_IF\\_setAuthCommands\(\)](#),  
[RMAP\\_TARGET\\_PXI\\_IF\\_getAuthCommands\(\)](#)

These functions can be used to set and get the authorised command types to authorise or reject an incoming RMAP command based on the command type in the RMAP command header. The set function takes a device identifier, a target index and a mask describing the commands to be authorised. The get function takes a device identifier, a target index and a pointer to a value which will be updated with a mask describing the commands authorised.

- [RMAP\\_TARGET\\_PXI\\_IF\\_setAuthKeyRange\(\)](#),  
[RMAP\\_TARGET\\_PXI\\_IF\\_getAuthKeyRange\(\)](#)

These functions can be used to set and get the authorised key range to authorise or reject an incoming RMAP command based on the key value in the RMAP command header. The set function takes a device identifier, a target index, and the values of the lowest and highest keys authorised. The get function takes a device identifier, a target index, and two pointers to values which will be updated with the lowest and highest keys authorised.



- `RMAP_TARGET_PXI_IF_setAuthMemoryAddressRange()`,  
`RMAP_TARGET_PXI_IF_getAuthMemoryAddressRange()`

These functions can be used to set and get the authorised memory address range to authorise or reject an incoming RMAP command based on the memory address value in the RMAP command header. The set function takes a device identifier, a target index, and the values of the lowest and highest memory addresses authorised. The get function takes a device identifier, a target index, and two pointers to values which will be updated with the lowest and highest memory addresses authorised.

The example below shows how to set the authorisation mode to automatic and set each of the authorisation parameters for RMAP target 0 then read them back to confirm the changes. The authorisation parameters are configured to accept RMAP commands with a logical address of 32-63, a protocol ID of 1, a command type of RMAP read or write with reply, a key of 50-70 and an address of 0x00000020-0x00010000.

Note, for brevity, error checking for the API calls is not included in this example.

```
U8 logicalLow, logicalHigh;
U8 pid;
TARGET_AUTH_CMD_MASK commands;
U8 keyLow, keyHigh;
U32 memoryLow, memoryHigh;

int authModeSetSuccess
int laSetSuccess, laGetSuccess;
int pidSetSuccess, pidGetSuccess;
int cmdSetSuccess, cmdGetSuccess;
int keySetSuccess, keyGetSuccess;
int memSetSuccess, memGetSuccess;

/* Set the authorisation mode to automatic */
authModeSetSuccess = RMAP_TARGET_PXI_IF_setAuthControlMode(deviceId, 0,
    TARGET_AUTH_MODE_AUTOMATIC);

/* Set the authorised logical address range */
laSetSuccess = RMAP_TARGET_PXI_IF_setAuthLogicalAddressRange(deviceId, 0,
    32, 63);

/* Set the authorised protocol ID */
pidSetSuccess = RMAP_TARGET_PXI_IF_setAuthProtocolId(deviceId, 0, 1);

/* Set the authorised commands */
cmdSetSuccess = RMAP_TARGET_PXI_IF_setAuthCommands(deviceId, 0,
    TARGET_AUTH_CMD_READ | TARGET_AUTH_CMD_WRITE_REPLY);

/* Set the authorised key range */
keySetSuccess = RMAP_TARGET_PXI_IF_setAuthKeyRange(deviceId, 0, 50, 70);

/* Set the authorised memory address range */
memSetSuccess = RMAP_TARGET_PXI_IF_setAuthMemoryAddressRange(deviceId, 0,
    0x00000020, 0x00010000);

/* Get the authorised logical address range */
laGetSuccess = RMAP_TARGET_PXI_IF_getAuthLogicalAddressRange(deviceId, 0,
    &logicalLow, &logicalHigh);
```



```

/* Get the authorised protocol ID */
pidGetSuccess = RMAP_TARGET_PXI_IF_getAuthProtocolId(deviceId, 0, &pid);

/* Get the authorised commands */
cmdGetSuccess = RMAP_TARGET_PXI_IF_getAuthCommands(deviceId, 0, &commands);

/* Get the authorised key range */
keyGetSuccess = RMAP_TARGET_PXI_IF_getAuthKeyRange(deviceId, 0,
    &keyLow, &keyHigh);

/* Get the authorised memory address range */
memGetSuccess = RMAP_TARGET_PXI_IF_getAuthMemoryAddressRange(deviceId, 0,
    &memoryLow, &memoryHigh);

/* Compare the parameters with the expected values */
printf("The authorised logical address range for target 0 was %sset correctly.\n",
    (logicalLow == 32 && logicalHigh == 63) ? "" : "not ");
printf("The authorised protocol ID for target 0 was %sset correctly.\n",
    pid == 1 ? "" : "not ");
printf("The authorised commands for target 0 was %sset correctly.\n",
    commands == (TARGET_AUTH_CMD_READ |
    TARGET_AUTH_CMD_WRITE_REPLY) ? "" : "not ");
printf("The authorised key range for target 0 was %sset correctly.\n",
    (keyLow == 50 && keyHigh == 70) ? "" : "not ");
printf("The authorised memory address range for target 0 was %sset correctly.\n",
    (memoryLow == 0x00000020 && memoryHigh == 0x00010000 ? "" : "not ");

```

## Performing Manual RMAP command Authorisation

The RMAP Target API provides the following functions for performing manual RMAP command authorisation:

- **RMAP\_TARGET\_PXI\_IF\_isAuthRequired()**  
This function can be used to determine if there is an RMAP command waiting to be authorised or rejected by the user application, if the authorisation mode is set to manual. It takes a device identifier and a target index and returns 1 if there is a command awaiting authorisation, or 0 if there is no waiting command.
- **RMAP\_TARGET\_PXI\_IF\_getWaitingCommand()**  
This function can be used to read the parameters of the RMAP command awaiting authorisation by the user application. It takes a device identifier, a target index and a pointer to a data structure that will be updated with the waiting RMAP command's parameters.
- **RMAP\_TARGET\_PXI\_IF\_authoriseWaitingCommand()**  
This function can be used to authorise the RMAP command awaiting authorisation by the user application. It takes a device identifier and a target index.
- **RMAP\_TARGET\_PXI\_IF\_rejectWaitingCommand()**  
This function can be used to reject the RMAP command awaiting authorisation by the user application. It takes a device identifier, a target index and a reason for rejection which is used to form the corresponding RMAP reply status in the RMAP reply packet returned to the initiator.

The example below shows how to set the authorisation mode to manual then wait for an incoming RMAP command for RMAP target 0 that requires authorisation which is read then authorised or rejected based on the incoming RMAP command's parameters. The example will accept the RMAP command if it has a logical address of 32-63, a protocol ID of 1, a command type of RMAP read or write with reply, a key of 50-70 and an address of 0x00000020-0x00010000.





Note, for brevity, error checking for the API calls is not included in this example.

```
int authModeSetSuccess;
int getWaitingCommandSuccess;
RmapCommandParameters waitingCommand;
U16 tid;

/* Set the authorisation mode to manual */
authModeSetSuccess = RMAP_TARGET_PXI_IF_setAuthControlMode(deviceId, 0,
    TARGET_AUTH_MODE_MANUAL);

/* Wait for a command that requires authorisation */
while(!RMAP_TARGET_PXI_IF_isAuthRequired(deviceId, 0));

/* Get the waiting command's parameters */
getWaitingCommandSuccess = RMAP_TARGET_PXI_IF_getWaitingCommand(deviceId, 0,
    &waitingCommand);

/* Get the waiting command's transaction ID */
tid = waitingCommand.transactionId;

/* Check the waiting command's logical address */
if(waitingCommand.targetLogicalAddress < 32 ||
    waitingCommand.targetLogicalAddress > 63)
{
    printf("The command (%u) has an invalid logical address.\n", tid);
    RMAP_TARGET_PXI_IF_rejectWaitingCommand(deviceId, 0,
        REJECTION_REASON_LOGICAL_ADDRESS);
    return;
}

/* Check the waiting command's protocol ID */
if(waitingCommand.protocolId != 1)
{
    printf("The command (%u) has an invalid protocol ID.\n", tid);
    RMAP_TARGET_PXI_IF_rejectWaitingCommand(deviceId, 0,
        REJECTION_REASON_PROTOCOL_ID);
    return;
}

/* Check the waiting command's command type */
if(waitingCommand.command != TARGET_AUTH_CMD_READ &&
    waitingCommand.command != TARGET_AUTH_CMD_WRITE_REPLY)
{
    printf("The command (%u) has an invalid command type.\n", tid);
    RMAP_TARGET_PXI_IF_rejectWaitingCommand(deviceId, 0,
        REJECTION_REASON_OTHER);
    return;
}

/* Check the waiting command's key */
if(waitingCommand.key < 50 || waitingCommand.key > 70)
{
    printf("The command (%u) has an invalid key.\n", tid);
    RMAP_TARGET_PXI_IF_rejectWaitingCommand(deviceId, 0,
        REJECTION_REASON_KEY);
    return;
}
```



```

/* Check the waiting command's memory address */
if(waitingCommand.address < 0x00000020 ||
    waitingCommand.address > 0x00010000)
{
    printf("The command (%u) has an invalid memory address.\n", tid);
    RMAP_TARGET_PXI_IF_rejectWaitingCommand(deviceId, 0,
        REJECTION_REASON_OTHER);
    return;
}

/* The command has passed all checks, so authorise it */
printf("The command (%u) is valid.\n", tid);
RMAP_TARGET_PXI_IF_authoriseWaitingCommand(deviceId, 0);

```

## Enabling and Receiving Notifications

The RMAP Target API provides the following functions for configuring RMAP command activity notifications:

- RMAP\_TARGET\_PXI\_IF\_setEnabledNotifications(),**  
**RMAP\_TARGET\_PXI\_IF\_getEnabledNotifications()**  
 These functions can be used to set and get the types of notifications that are enabled. The set function takes a device identifier, a target index and a mask describing the notifications to be enabled. The get function takes a device identifier, a target index and a pointer to a value that will be updated with a mask describing the enabled notifications.
- RMAP\_TARGET\_PXI\_IF\_registerNotificationListener(),**  
**RMAP\_TARGET\_PXI\_IF\_unregisterNotificationListener()**  
 These functions can be used to register and unregister the listener function that will be called whenever a notification type is received. The register function takes a device identifier, a target index, a notification type and a pointer to a listener function. The unregister function takes a device identifier, a target index and a notification type.
- RMAP\_TARGET\_PXI\_IF\_startReceivingNotifications(),**  
**RMAP\_TARGET\_PXI\_IF\_stopReceivingNotifications()**  
 These functions can be used to start and stop the receiving of notifications for a device. The set and get functions both take a device identifier.

The example below shows how to enable RMAP command notifications and start receiving notifications which will cause a registered listener function to be called each time a notification is received.

Note, for brevity, error checking for the API calls is not included in this example.

```

void STAR_API_CC commandCompleteListener(U32 target, void *pContext)
{
    printf("Command notification for target %u.\n", target);

    /* Perform any processing on the notification packet stored in pContext */
}

int main(int argc, char **argv)
{
    STAR_DEVICE_ID deviceId;
    int enableSuccess;
    int registerSuccess;

```



```

int startSuccess;

/* Select a device */
deviceId = selectDevice();

/* Enable command complete notifications for target 0 */
enableSuccess = RMAP_TARGET_PXI_IF_setEnabledNotifications(deviceId, 0,
    NOTIF_TYPE_CMD_COMPLETE);

/* Register command complete notification listener for target 0 */
registerSuccess = RMAP_TARGET_PXI_IF_registerNotificationListener(deviceId,
    0, NOTIF_TYPE_CMD_COMPLETE, commandCompleteListener);

/* Start receiving notifications */
startSuccess = RMAP_TARGET_PXI_IF_startReceivingNotifications(deviceId);

/* Spin and wait for notifications */
while(true);
}

```

## Reading and Writing to Memory

The RMAP Target API provides the following functions for reading and writing to the target memory:

- **RMAP\_TARGET\_PXI\_IF\_readMemory()**  
This function can be used to read from target memory. It takes a device identifier, a target index, a memory address, a length and a pointer to a buffer that the target memory will be read into.
- **RMAP\_TARGET\_PXI\_IF\_writeMemory()**  
This function can be used to write to target memory. It takes a device identifier, a target index, a memory address, a length and a pointer to a buffer that will be written into the target memory.

The example below shows how to write an incrementing buffer of data to address 0x00000000 in target 0's memory then read it back to confirm that the write was successfully executed.

Note, for brevity, error checking for the API calls is not included in this example.

```

U8 writeBuffer[1024], readBuffer[1024];
int writeSuccess, readSuccess;
int errors;
int i;

/* Initialise the write buffer to incrementing bytes */
for(i = 0; i < 1024; ++i)
{
    writeBuffer[i] = i;
}

/* Clear the read buffer */
memset(readBuffer, 0, 1024);

/* Write to memory */
writeSuccess = RMAP_TARGET_PXI_IF_writeMemory(deviceId, 0, 0x00000000,
    1024, writeBuffer);

/* Read from memory */

```



```
readSuccess = RMAP_TARGET_PXI_IF_readMemory(deviceId, 0, 0x00000000,
    1024, readBuffer);

/* Count any errors within the buffers */
errors = 0;
for(i = 0; i < 1024; ++i)
{
    if(writeBuffer[i] != readBuffer[i]) ++errors;
}

/* Print the number of errors, if any */
if(!errors)
{
    printf("The write was successful.\n");
}
else
{
    printf("There were %d errors.\n", errors);
}
```



**STAR-Dundee**

STAR-Dundee  
STAR House, 166 Nethergate  
Dundee, DD1 4EE, Scotland, UK  
Tel: +44 1382 201 755 Fax: +44 1382 388 838  
Email: [enquiries@star-dundee.com](mailto:enquiries@star-dundee.com)  
Web: [www.star-dundee.com](http://www.star-dundee.com)